AUTOMATED TEST CONTROLLER
GUIDE MANUAL

CREATED BY : AHMAD
MESURO LTD
PENCOED TECHNOLOGY PARK, PENCOED, CF35 5HZ, UK

INTRODUCTION

This document contains information regarding the Automation Test Controller. The framework is a modular framework designed to streamline and standardise RF device characterization through automated measurement workflows. It supports both S-parameter and Load Pull measurements, integrating seamlessly with external instrumentation via the RapidMS Client API.

The framework was developed to address the following goals :

1) Provide reproducible measurement results by automating traditionally manual processes
2) Reduce test time through automated file parsing, multi-test execution and multi-threaded test runs
3) Support different RF test types within a unified framework
4) Offer a desktop GUI and a web-based results interface for savable reports

At a high level the project is composed of :

1) Core data models (ClassFile.py) for representing S-parameter and load pull indices
2) Test automation modules (Sparameter_Automation.py and Load_Pull.py) for executing measurement workflows
3) A centralised controller (main.py) that provides the GUI, manages test execution and displays results

ClassFile	6/24/2025 3:39 PM	Python Source File	5 KB
Load_Pull	8/12/2025 10:45 AM	Python Source File	32 KB
main	8/27/2025 2:05 PM	Python Source File	56 KB
Sparameter_Automation	8/27/2025 2:05 PM	Python Source File	16 KB

By combining automated hardware integration and user-friendly interfaces, this project enables engineers to accelerate RF validation cycles and improve overall measurement reliability.

BEGINNERS GUIDE -

The Automation Test Controller is built to help engineers start testing quickly with minimal setup. This section walks through the basics of how to prepare and run tests

STEP 1 : PREPARE TEST DATA

The framework uses CSV configuration files to define which tests to run and under what conditions. Each row in the file corresponds to one test case :

Test_No	Test_Type	Min_Freq	Max_Freq	Freq_Step	Min_Pwr	Max_Pwr
1	s parameter single	2.3	2.3	0	0	0

Here is an example of some test parameters that can be changed. There are many more customisations that can be done for each test.

STEP 2 : LAUNCH TEST CONTROLLER

Run the application by executing the main.py file “python main.py” in the terminal. This will lead to the desktop GUI appearing with an interface to :

1) Load the CSV configuration file
2) Filter the type of tests to run from the CSV file and which test types to avoid
3) Start and stop the test runs
4) Open and view the web report of test results from the test-suite

STEP 3 : RUNNING A TEST

1) Load the CSV file using the “Load CSV” button in the GUI
2) Select the test types you'd like to run from the test suite (select all if you want to run all tests from the CSV).
3) Start the test, the system automatically connects to the hardware and executes the workflow
4) Progress and results are displayed in real time

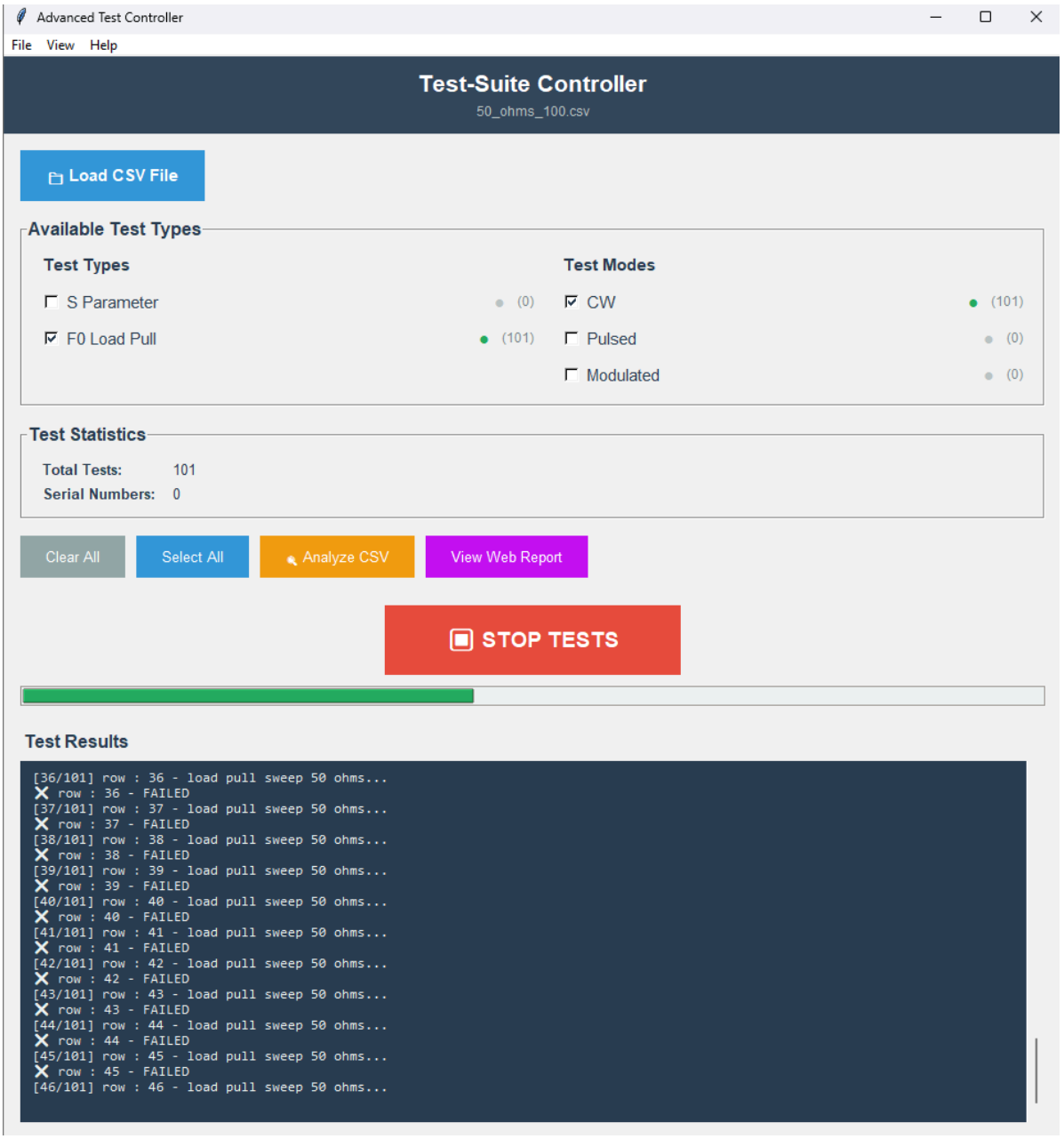
STEP 4 : REVIEW RESULTS

Results are visible directly in the GUI once each test completes, However, for a more detailed report, a built in web server provides a browser based view of the results. This should enable users to access test outcomes and review the measurement metrics and files

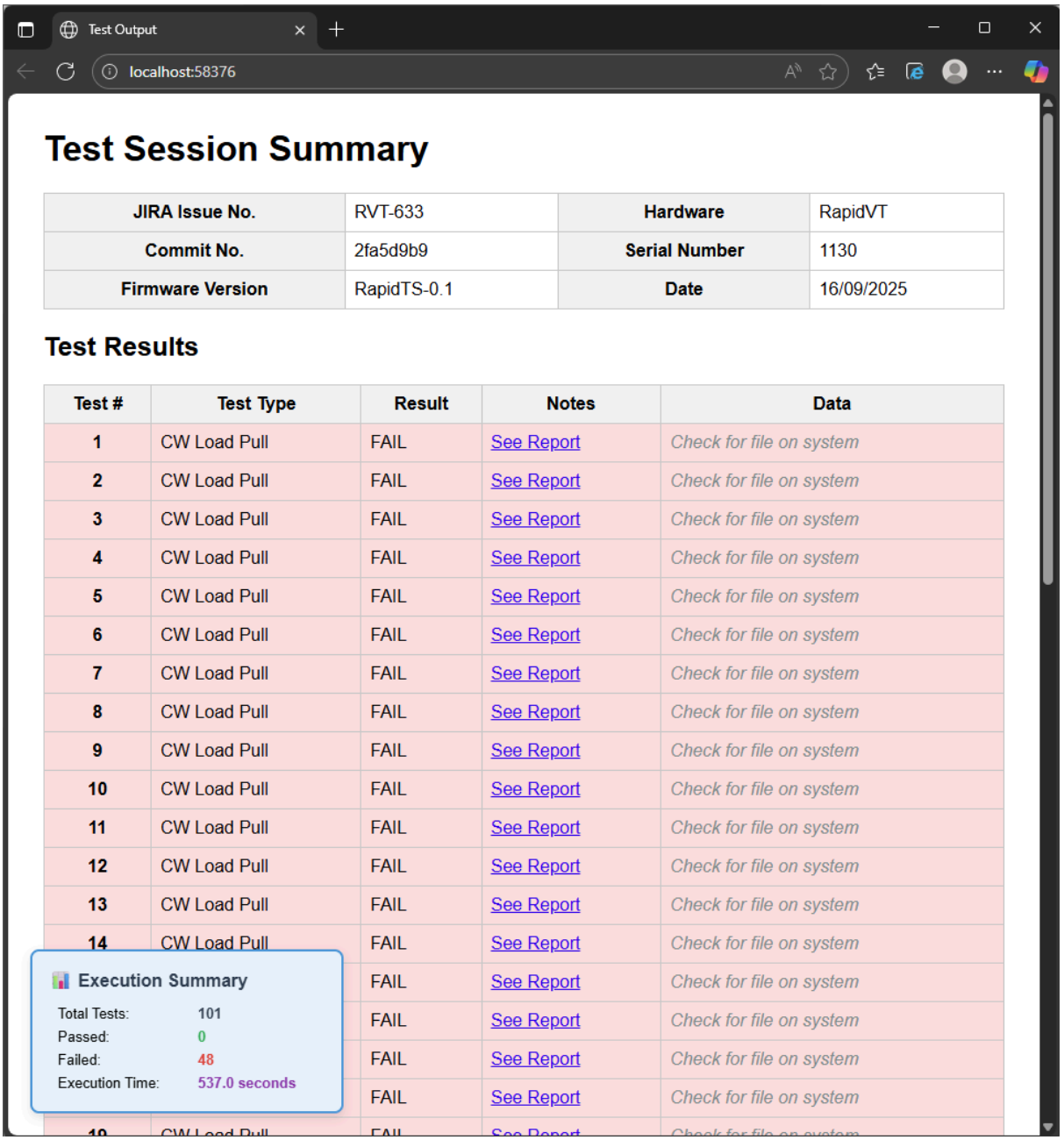
SCREENSHOTS -

Below you will find screenshots of some of the components and scenarios of how the automated test controller works.

TEST PANEL :



Test Output Web Report :



Test Output

localhost:58376

Test Session Summary

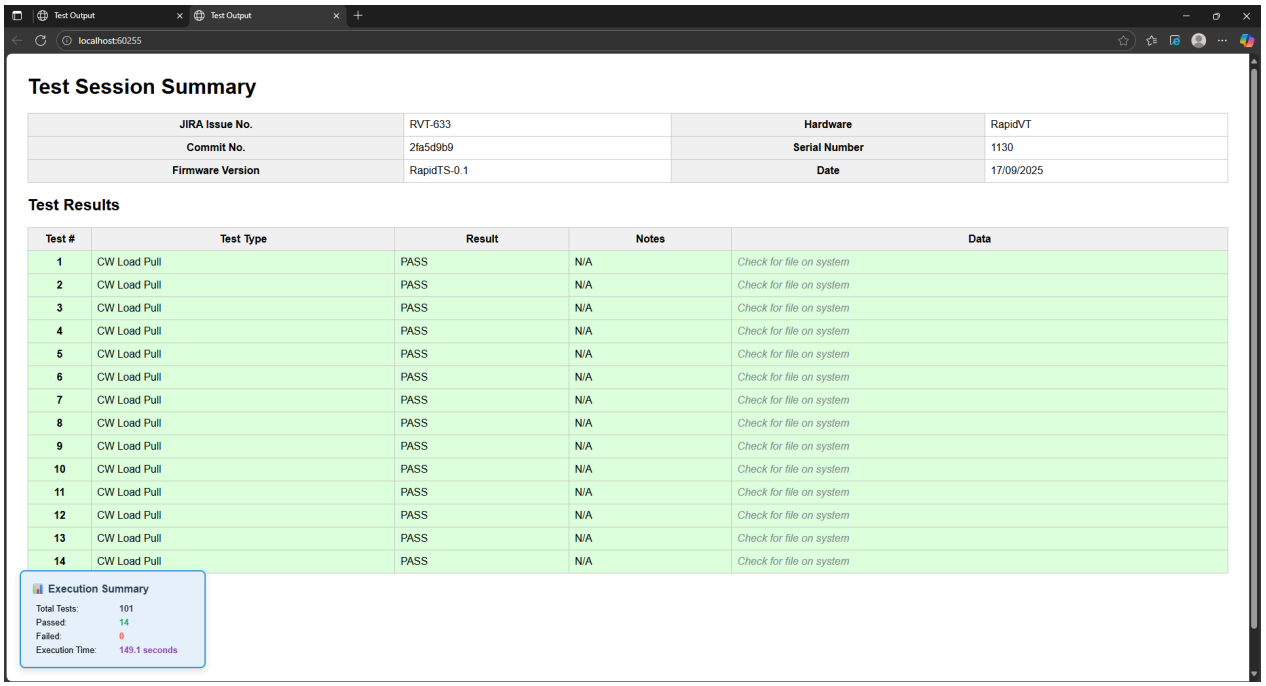
JIRA Issue No.	RVT-633	**Hardware**	RapidVT
Commit No.	2fa5d9b9	**Serial Number**	1130
Firmware Version	RapidTS-0.1	**Date**	16/09/2025

Test Results

Test #	Test Type	Result	Notes	Data
1	CW Load Pull	FAIL	See Report	*Check for file on system*
2	CW Load Pull	FAIL	See Report	*Check for file on system*
3	CW Load Pull	FAIL	See Report	*Check for file on system*
4	CW Load Pull	FAIL	See Report	*Check for file on system*
5	CW Load Pull	FAIL	See Report	*Check for file on system*
6	CW Load Pull	FAIL	See Report	*Check for file on system*
7	CW Load Pull	FAIL	See Report	*Check for file on system*
8	CW Load Pull	FAIL	See Report	*Check for file on system*
9	CW Load Pull	FAIL	See Report	*Check for file on system*
10	CW Load Pull	FAIL	See Report	*Check for file on system*
11	CW Load Pull	FAIL	See Report	*Check for file on system*
12	CW Load Pull	FAIL	See Report	*Check for file on system*
13	CW Load Pull	FAIL	See Report	*Check for file on system*
14	CW Load Pull	FAIL	See Report	*Check for file on system*
		FAIL	See Report	*Check for file on system*
		FAIL	See Report	*Check for file on system*
		FAIL	See Report	*Check for file on system*
		FAIL	See Report	*Check for file on system*

Execution Summary

Total Tests:	101
Passed:	0
Failed:	48
Execution Time:	537.0 seconds



Test Session Summary

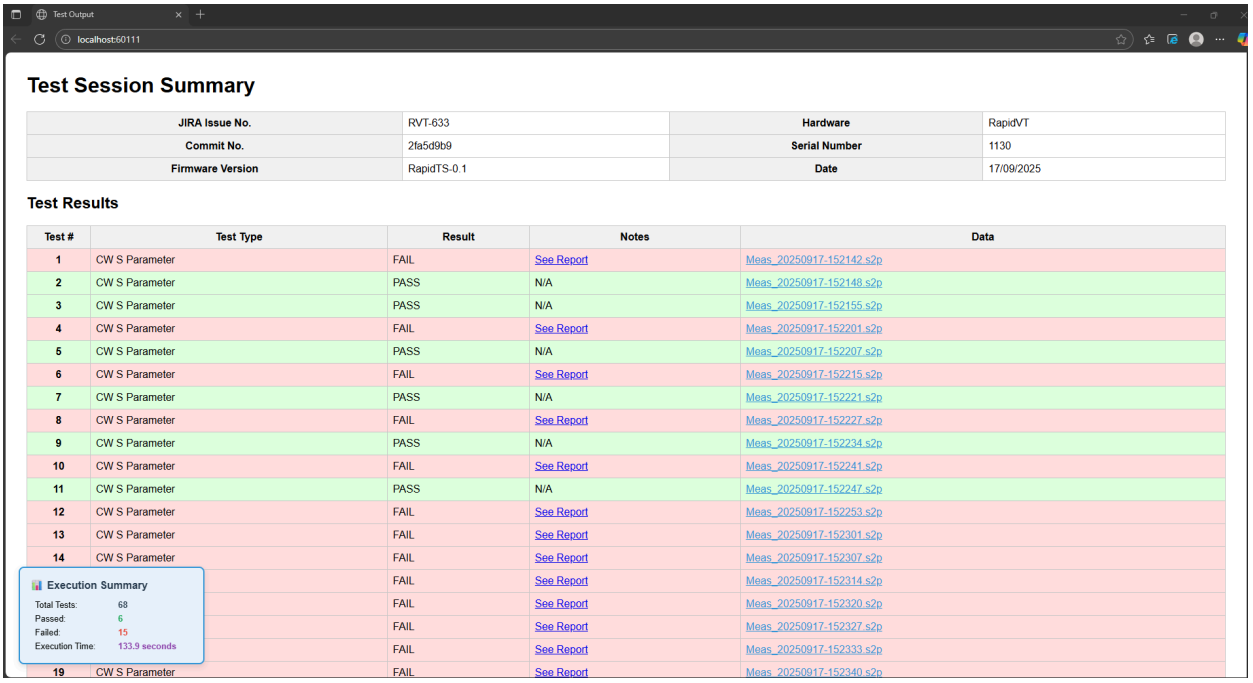
JIRA Issue No.	RVT-633	**Hardware**	RapidVT
Commit No.	2fa5d9b9	**Serial Number**	1130
Firmware Version	RapidTS-0.1	**Date**	17/09/2025

Test Results

Test #	Test Type	Result	Notes	Data
1	CW Load Pull	PASS	N/A	*Check for file on system*
2	CW Load Pull	PASS	N/A	*Check for file on system*
3	CW Load Pull	PASS	N/A	*Check for file on system*
4	CW Load Pull	PASS	N/A	*Check for file on system*
5	CW Load Pull	PASS	N/A	*Check for file on system*
6	CW Load Pull	PASS	N/A	*Check for file on system*
7	CW Load Pull	PASS	N/A	*Check for file on system*
8	CW Load Pull	PASS	N/A	*Check for file on system*
9	CW Load Pull	PASS	N/A	*Check for file on system*
10	CW Load Pull	PASS	N/A	*Check for file on system*
11	CW Load Pull	PASS	N/A	*Check for file on system*
12	CW Load Pull	PASS	N/A	*Check for file on system*
13	CW Load Pull	PASS	N/A	*Check for file on system*
14	CW Load Pull	PASS	N/A	*Check for file on system*

Execution Summary

Total Tests: 101
Passed: 14
Failed: 0
Execution Time: 149.1 seconds



Test Session Summary

JIRA Issue No.	RVT-633	**Hardware**	RapidVT
Commit No.	2fa5d9b9	**Serial Number**	1130
Firmware Version	RapidTS-0.1	**Date**	17/09/2025

Test Results

Test #	Test Type	Result	Notes	Data
1	CW S Parameter	FAIL	See Report	Meas_20250917-152142.s2p
2	CW S Parameter	PASS	N/A	Meas_20250917-152148.s2p
3	CW S Parameter	PASS	N/A	Meas_20250917-152155.s2p
4	CW S Parameter	FAIL	See Report	Meas_20250917-152201.s2p
5	CW S Parameter	PASS	N/A	Meas_20250917-152207.s2p
6	CW S Parameter	FAIL	See Report	Meas_20250917-152215.s2p
7	CW S Parameter	PASS	N/A	Meas_20250917-152221.s2p
8	CW S Parameter	FAIL	See Report	Meas_20250917-152227.s2p
9	CW S Parameter	PASS	N/A	Meas_20250917-152234.s2p
10	CW S Parameter	FAIL	See Report	Meas_20250917-152241.s2p
11	CW S Parameter	PASS	N/A	Meas_20250917-152247.s2p
12	CW S Parameter	FAIL	See Report	Meas_20250917-152253.s2p
13	CW S Parameter	FAIL	See Report	Meas_20250917-152301.s2p
14	CW S Parameter	FAIL	See Report	Meas_20250917-152307.s2p
		FAIL	See Report	Meas_20250917-152314.s2p
		FAIL	See Report	Meas_20250917-152320.s2p
		FAIL	See Report	Meas_20250917-152327.s2p
		FAIL	See Report	Meas_20250917-152333.s2p
19	CW S Parameter	FAIL	See Report	Meas_20250917-152340.s2p

Execution Summary

Total Tests: 68
Passed: 6
Failed: 15
Execution Time: 133.9 seconds

Load Pull Comparison Report

localhost:58376/report/0

Load Pull Comparison Report: FAILED

Test Configuration

Mode	CW	**Source Cal File**	C:\Users\NIK_Demo\Desktop\Ahmad\Test_12-09-2025\spc.csv
IF Bandwidth	1000 kHz	**Vector Cal File**	C:\Users\NIK_Demo\Desktop\Ahmad\Test_12-09-2025\VectorCorrectionFull.csv
Frequency	1.8 GHz	**Initialization File**	C:\Users\NIK_Demo\Desktop\Ahmad\Test_12-09-2025\INIT.jinit

Measurement Point

Power: -10.0 dBm **Target Gamma:** 0.0 **Target Phase:** 0.0°

Parameter	Gold Standard	Measured	Difference	Status
PhiLWaves@F0[deg]	-158.588	-149.190	9.400 (tol = 1.500)	FAIL
|GLWaves@F0|	0.006	0.131	0.120 (tol = 0.010)	FAIL
GainWavesPwr[dB]	-0.002	0.200	0.200 (tol = 0.200)	FAIL

Measurement Point

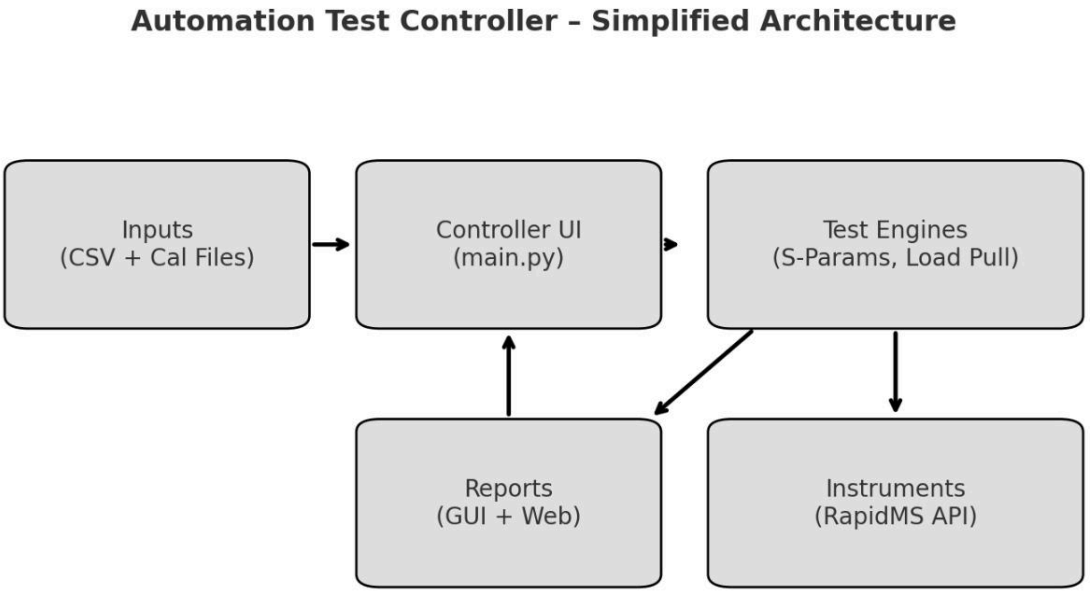
Power: -9.0 dBm **Target Gamma:** 0.0 **Target Phase:** 0.0°

Parameter	Gold Standard	Measured	Difference	Status
PhiLWaves@F0[deg]	-28.391	-149.190	120.800 (tol = 1.500)	FAIL
|GLWaves@F0|	0.003	0.131	0.130 (tol = 0.010)	FAIL
GainWavesPwr[dB]	-0.001	0.200	0.200 (tol = 0.200)	FAIL

Measurement Point

FRAMEWORK ARCHITECTURE -

The Automation Test Controller is designed around a modular architecture that separates data handling, test execution and user interaction. This makes the framework easier to maintain, extend and increase scalability.



1) Data Models

- Defined in ClassFile.py
- Provide structured representations for S-parameters and Load Pull indices
- Ensure consistency when parsing measurement results and reference data

2) Test Execution Modules

- Sparameter_Automation.py handles automated S-parameter testing
- Load_Pull.py manages automated load pull workflows
- Both integrate with the RapidMS Client API to control RF Test equipment
- Support parsing of gold standard reference files for automatic result comparison

3) File placement

- Calibration files - Test System
- Initialisation File - Test System
- Reference Data file - Local System
- Impedance Grid file - Local System

4) Controller and Interfaces

- main.py provides the desktop GUI (Tkinter) for test setup and execution
- A lightweight web server (Flask) runs alongside the GUI for remote access to results
- Multi-threaded execution ensures tests run without blocking the interface

5) Results and Reporting

- Results are accessible briefly in the GUI and in further detail in the web dashboard
- Support for pass/fail verdicts and structured logging

CONFIGURATION AND CUSTOMISATION -

Certain parameters can be adjusted to suit specific test environments and requirements. This section outlines the most common areas where users may need to make changes

1) Measurement Tolerances : By default, the system evaluates measured results against reference files. This comparison uses magnitude and phase tolerances to determine whether a test passes or fails

    - Magnitude Tolerance defines the acceptable deviation in dB
    - Phase Tolerance defines the acceptable deviation in degrees
    - An area in the code as shown below checks if the signal level is low and sets a personalised hardcoded tolerance for that as well

Sparameter_Automation.py :

The code snippet below can be found in the run_s_param_test() method

```
# Process and analyse results
measured = load_measurement_data(measurement_data)
file_path = output_measured_data(measured)
gold_standard = load_gold_standard(gs)
filtered_gs, filtered_meas = process_both(gold_standard, measured)
verdict, err_dict, mag_tol, phase_tol = compare(filtered_gs, filtered_meas, 0.2, 3)
```

The code snippet below can be found in the compare() method

```
            # Adjust tolerance for low signal levels
            if mm < -40:
                mag_tolerance = 0.6
                phase_tolerance = 5

            mag_diff = abs(gm - mm)

            # Normalise phase
            phase1_norm = gp % 360
            phase2_norm = mp % 360
            phase_diff = abs(phase1_norm - phase2_norm)
            if phase_diff > 180:
                phase_diff = 360 - phase_diff

```

Load_Pull.py :

The snippet below can be found in the run_test() method

```
        # Compare results against gold standard
        if bias_control:
            if ismod:
                comparison_parameters = [
                ('PhiLoad@F0[deg]', 1.5),      # Phase tolerance: 1 degree
                ('|GLoad@F0|', 0.01),        # Gamma magnitude tolerance: 0.01
                ('OutputEff[%]', 0.2),         # Efficiency tolerance: 0.2%
                ('GainPwr[dB]', 0.3)      # Gain tolerance: 0.2 dB
                ]
            else:
                comparison_parameters = [
                    ('PhiLWaves@F0[deg]', 1.5),      # Phase tolerance: 1 degree
                    ('|GLWaves@F0|', 0.01),        # Gamma magnitude tolerance: 0.01
                    ('OutputEff[%]', 0.2),         # Efficiency tolerance: 0.2%
                    ('GainWavesPwr[dB]', 0.2)      # Gain tolerance: 0.2 dB
                ]
        else:
            if ismod:
                comparison_parameters = [
                ('PhiLoad@F0[deg]', 1.5),      # Phase tolerance: 1 degree
                ('|GLoad@F0|', 0.01),        # Gamma magnitude tolerance: 0.01
                ('GainPwr[dB]', 0.3)      # Gain tolerance: 0.2 dB
                ]
            else:
                comparison_parameters = [
                    ('PhiLWaves@F0[deg]', 1.5),      # Phase tolerance: 1 degree
                    ('|GLWaves@F0|', 0.01),        # Gamma magnitude tolerance: 0.01
```

These tolerances are hardcoded in the Sparameter_Automation.py and Load_Pull.py. Below is a screenshot of where they are in the code. This can be added as a configuration to the CSV as an improvement

2) Hardware TestSet IP Configuration : The framework communicates with the RF hardware over the network. The IP address of the hardware testset must be specified so the automation framework can connect and run the tests on the relevant system

    - This value is provided in the CSV file under the relevant column
    - Each test case can reference a specific hardware IP, allowing multiple systems and setups to be managed centrally

IP	Mode	InputV	OutputV	Aux1V	Aux2V
192.168.28.209	CW				

3) Other Configurable Parameters :

    - Frequency ranges (s-parameter)
    - Power Sweeps
    - Impedance Sweeps
    - Bias control
    - Calibration files for source phase calibration and vector calibration
    - Reference data
    - Mode (CW, Pulsed and Modulated)
    - Voltages
    - Sweep stop conditions (i.e compression)
    - Bandwidth
    - IF Bandwidth
    - Modulated Bandwidth
    - Waveform file selection
    - Pulsed Settings

In the future, more configurable parameters can be added such as de-embedding files, LO cal files etc…

PREREQUISITES AND INSTALLATION -

Before running the automation framework, make sure your system has the required software and libraries installed.

The project includes a conda environment YAML file (API_Meas.yaml) that contains all the dependencies to run the framework :

- Core Python libraries (pandas, numpy, TKinter, flask etc…)
- .NET interoperability (pythonnet) for the RapidMS API
- Additional utilities for logging, threading and file handling

Here is a step by step setup guide :

Step 1 : Download Anaconda Navigator if not already available

Step 2: Create the environment

Open anaconda prompt or a terminal and navigate to the project folder. Once there, run :

“conda create -f environment.yml”

Step 3 : Activate the environment

“conda activate API_Meas”

Additionally there is another way we can achieve this through the requirements.txt :

Step 1 : Create a virtual environment : “`python -m venv {environmentname}`

Step 2 : Activate the environment : “`path\to\venv\Scripts\activate”`

Step 3 : Ensure the requirements.txt file is present in the directory where the python scripts and virtual environment is. Once that's done open the directory in command prompt and type the command “`pip install -r requirements.txt”`

Step 4 : Run the automation framework using “`python main.py”`

RAPIDMS API SETUP -

The system requires the RapidMS Client API DLL to communicate with hardware

- Ensure that RapidMS.ClientAPI.dll is available in the same path as the scripts

Name	Date modified	Type	Size
templates	9/19/2025 11:53 AM	File folder	
API_Meas.yaml	7/9/2025 9:56 AM	Yaml Source File	19 KB
ClassFile.py	6/24/2025 3:39 PM	Python Source File	5 KB
Load_Pull.py	9/19/2025 3:33 PM	Python Source File	34 KB
main.py	9/16/2025 10:02 AM	Python Source File	56 KB
requirements.txt	9/17/2025 1:21 PM	TXT File	1 KB
Sparameter_Automation.py	9/19/2025 3:34 PM	Python Source File	16 KB
Template.csv	9/15/2025 3:47 PM	CSV File	1 KB
RapidMS.ClientAPI.dll	5/29/2025 10:17 AM	Application extens...	35 KB
RapidMS.ClientAPI.dll.config	8/13/2024 9:44 AM	CONFIG File	1 KB

- If this path differs in your setup, please update the DLL path in both Sparameter_Automation.py and Load_Pull.py